

Technical Approach — Grounded Answer Engine (RAG)

The retrieval-augmented generation pipeline that guarantees every legal answer is traceable to a source passage — with code.

1. Technologies and Libraries Used

Component	Choice	Role
Runtime	Node.js 24 + TypeScript	API server (Express), WebSocket server (ws)
LLM provider	Groq (OpenAI-compatible API)	<code>openai/gpt-oss-120b</code> primary, <code>openai/gpt-oss-20b</code> voice-first + fallback
Retrieval	Custom BM25 (no external lib)	Explainable keyword search, deterministic ordering
Database	Supabase (PostgreSQL)	<code>corpus_passages</code> , sessions, messages, cases, facts, grievances
Citation parsing	Custom regex (<code>src/utils/citation.ts</code>)	Extracts <code>[Source: ...]</code> tags; bare- <code>Source:</code> pattern guarded by <code>(?!\[)</code> lookbehind to avoid double-matching
Metrics	prom-client	<code>queriesTotal</code> , latency histogram, <code>fallbackTotal</code> , <code>groundingRetriesTotal</code> , <code>groundingFailuresTotal</code>

2. Step-by-Step Breakdown

Step 2.1 — Keyword extraction with Hinglish expansion

A rural user asks in Hinglish: *“member banein ke liye kya karna hoga?”*. The retriever normalizes, strips ~70 Hindi stop-words (`mai`, `kya`, `hai`, `ka`, `ki...`), builds bigrams, and expands keywords

through the bidirectional Hinglish map — so the Roman query matches Devanagari passages and vice-versa.

```
// src/modules/rag/retriever.ts
const HINGLISH_MAP: Record<string, string[]> = {
  'member':      ['sadasya', 'nisarga'],
  'membership':  ['sadasyata', 'sadasya'],
  'eligible':     ['eligibility', 'yogya', 'competent'],
  'premium':     ['kiraya', 'bhugtan', 'payment'],
  'insurance':   ['bima', 'bima yojana'],
  'grievance':   ['shikayat', 'samasya'],
  'loan':        ['rin', 'karj', 'kharza'],
  // ... 60+ entries covering cooperative, PMFBY, PACS vocabulary
};

export function expandKeywords(keywords: string[]): string[] {
  const expanded = new Set(keywords);
  for (const kw of keywords) {
    const hindi = HINGLISH_MAP[kw];
    if (hindi) hindi.forEach(h => expanded.add(h.toLowerCase()));
    for (const [eng, hindiList] of Object.entries(HINGLISH_MAP)) {
      if (hindiList.some(h => h === kw)) expanded.add(eng); // reverse lookup
    }
  }
  return [...expanded];
}
```

Step 2.2 — BM25 scoring with phrase boosts

```
export function computeSimilarity(textLower, keywords, originalQuery) {
  const k1 = 1.2, b = 0.75; // Okapi BM25 constants
  let score = 0;
  for (const kw of keywords.singles) { // single-word terms
    const tf = countOccurrences(textLower, kw);
    const idf = weights.get(kw) ?? 1; // log(1+(N-df+0.5)/(df+0.5))
    score += idf * (tf * (k1 + 1)) / (tf + k1 * (1 - b + b * docLen / avgDocLen));
  }
  for (const bigram of keywords.bigrams) {
    if (textLower.includes(bigram)) score += 3.0; // exact bigram boost
  }
  const phrase = originalQuery.toLowerCase().trim().replace(/[^\w\s\u0900-\u097F]/g, '');
  if (phrase.length > 8 && textLower.includes(phrase)) score += 2.5; // full-phrase boost
  return score;
}
```

Results are filtered at `score ≥ 0.2`, ranked, and the top 5 are returned — ties broken by `ORDER BY id` so the same query always yields the same passages (reproducible, judge-friendly).

Step 2.3 — Guardrailed prompt assembly

The system prompt is built per channel (`buildSystemPrompt` in `src/modules/llm/groq.ts`). It packages: the retrieved passages (truncated to 700 chars text / 250 voice to stay under Groq free-tier TPM), case facts from the `[[FACT]]` memory, and the recent conversation history — then applies behavioral rules (natural spoken Hindi, empathy in own words, one follow-up question, and the verbatim out-of-domain boundary message).

```
// passage packaging (groq.ts)
const passageLimit = isVoiceCall ? 250 : 700;
const passagesText = hasPassages
  ? ctx.passages.map(p => {
      const trimmed = p.text.length > passageLimit ? p.text.slice(0, passageLimit) + '...' : p.text;
      return `[$${p.sourceDoc}, $${p.sectionRef}]:\n${trimmed}`;
    }).join('\n\n')
  : 'No database passages retrieved.';

// model selection: voice channels get the fast model first
const primaryModel = isVoiceCall ? env.GROQ_MODEL_FALLBACK : env.GROQ_MODEL_PRIMARY; // 20b / 1
const backupModel = isVoiceCall ? env.GROQ_MODEL_PRIMARY : env.GROQ_MODEL_FALLBACK;
// temperature: 0.7 voice (natural variation) / 0.3 text (citation exactness)
// max_tokens: 350 voice / 600 text
```

Step 2.4 — Grounding retry: citation enforcement loop

After the first generation, the answer must contain `[Source: ...]` citations if passages existed. If not, one corrective retry is issued (text channels only):

```
// groq.ts - callGroq()
let answer = postProcessAnswer(response.choices[0].message.content || '', isVoiceCall);
let citations = extractCitations(answer);

if (!isVoiceCall && ctx.passages.length > 0 && citations.length === 0
  && answer !== DECLINE && answer !== BOUNDARY_MESSAGE) {
  const retry = await client.chat.completions.create({
    model,
    messages: [...messages,
      { role: 'assistant', content: answer },
      { role: 'user', content: 'Your previous answer had no [Source: ...] citations. Answer again grounding every factual claim in the RETRIEVED PASSAGES, and end the answer with the citation [Source: <doc name>, <section ref>].' }],
    temperature: 0.2, max_tokens: 500,
  });
  const retryAnswer = postProcessAnswer(retry.choices[0].message.content || '', false);
  if (extractCitations(retryAnswer).length > 0) {
```

```
    answer = retryAnswer; citations = extractCitations(retryAnswer);
    groundingRetriesTotal.inc();
  }
}
// If the retry still has no citations, the caller replaces the answer with
// the verified-decline: 'Mere paas iska verified jawab nahi hai.'
```

Step 2.5 — Boundary canonicalization

The domain boundary message must survive model drift verbatim. `isBoundaryLike()` normalizes common Devanagari variants (सिर्फ→sirf , कानून→kanoon , I→. ...) and, if $\geq 85\%$ of the boundary words match, re-canonicalizes the output to the exact message:

```
const BOUNDARY_MESSAGE = 'Main Sahakar Setu hoon aur sirf sahkari kanoon, sarkari yojanaon, ' +
  'PACS sevaon, PMFBY, vittiya saksharta aur shikayat samadhan se judi jaankari de sakta hoon. ' +
  'Aap in vishayon mein kuch pooch sakte hain.';

function isBoundaryLike(text: string): boolean {
  const normalized = DEVANAGARI_VARIANTS.reduce(
    (t, [dv, roman]) => t.split(dv).join(roman), text.replace(/I/g, '.'));
  const hits = BOUNDARY_MESSAGE.toLowerCase().replace(/I/g, '.')
    .split(/[\s,.]+/).filter(Boolean)
    .filter(w => normalized.toLowerCase().split(/[\s,.]+/).includes(w)).length;
  return hits / totalBoundaryWords >= 0.85;
}
```

3. Technical Details and Justifications

Decision	Why
BM25 instead of pure vector embeddings in the critical path	Deterministic, debuggable, zero cold-start cost; Devanagari embeddings are inconsistent on free tiers, while BM25 + Hinglish expansion is predictable. Verified 13/13 guide queries hit the correct passage in top-3.
Passage truncation (700/250 chars)	Groq free tier is 8000 TPM / 200K TPD — truncation keeps prompts small, latency low, budget safe.
Per-channel temperature (0.3 text / 0.7 voice)	Text needs citation discipline; voice needs natural variation so repeated calls don't sound scripted.
Small model first on voice	Median voice turn < 2.5 s target — 20b answers faster; 120b remains the fallback for harder questions.

Grounding retry with corrective message	One cheap self-correction pass removes most citationless answers; if it fails, decline — never fabricate a source.
Post-processing pipeline	Strips preamble/signoff fluff ("Sure!", "Feel free..."), removes markdown/citations on voice, truncates trailing sentence fragments — answers arrive clean on every channel.

4. Conclusion

The engine is deliberately boring where it must be trustworthy — deterministic retrieval, enforced citations, a verbatim decline — and deliberately adaptive where members need warmth — Hinglish-aware keywords, natural spoken Hindi, per-channel temperature. The result: a multilingual assistant a judge can interrogate on a live phone call and verify every answer against its cited passage.